

JAVA

OBJEKTOVĚ ORIENTOVANÉ PROGRAMOVÁNÍ V PRAXI

MGR. MARTIN SULÍR



Obsah

1. Úvod do objektově orientovaného programování (teorie).....	4
Důvody použití OOP	4
Zapouzdření	5
Dědičnost	6
Polymorfismus	6
Zadání	7
2. Třída Žák	8
Naše první třída (class)	9
Komentáře k zápisu atributů	10
Náš první objekt.....	11
Zadání	12
3. Konstruktory	13
Pravidla pro tvorbu konstruktoru:.....	13
Přetížený konstruktor	14
Odstranění duplicitního kódu v rámci přetíženého konstrukturu	15
Zrušení implicitního konstrukturu	16
4. Datum a čas	17
Postup implementace LocalDate do našeho programu	17
Výpis data	18
5. Metody (teorie).....	20
Konstrukce hlavičky	20
Tělo metody	21
Dělení metod dle vstupů a výstupů.....	21
Příklady použití metod.....	21
Zadání	22
6. Metody pro výpis informací	23
Metoda vypisInfo.....	23
Metoda toString	24
Zadání	26
7. Tvorba konkrétních metod.....	27
Metoda: Změň příjmení	27
Metoda: Zvyš ročník.....	28
Metoda: Pohlaví.....	28
Metoda: Plnoletost	29

Zadání	30
8. Modifikátory přístupu, Gettry a Settry	31
Typy modifikátorů přístupu u proměnných či metod	31
Metody Get a Set	33
Zadání	34
9. Třída ArrayList (teorie)	35
Tvorba ArrayListu	35
Procházení ArrayListu	36
Předdefinované metody ArrayListu	37
10. Class Třída, metody pro výpis	39
Definování class Třída	39
Metody vypisující informace o třídě	40
Zadání	41
11. Další metody pro class Třída	42
Metoda: Je žák ve třídě	42
Metoda: Přidej nového žáka	43
Metoda: Odeber žáka	46
Metoda: Zvyš ročník	47
Zadání	48
12. Validace rodného čísla	49
Formát rodného čísla	49
Tvorba statického interface	49
Metoda: Rodné číslo - kontrola	50
Metoda: Ověř formát rodného čísla	52
Metoda: Ověř dělitelnost 11ti	53
Metoda: Ověř měsíc narození z rodného čísla	53
Metoda: Ověř den narození z rodného čísla	54
Metoda: Ověř rok narození z rodného čísla	54
Implementace a otestování metody rodneCisloKontrola	54
Volání a ověřování metody pro přidávání nového žáka	55
13. Dědičnost	57
Dědický vztah Osoba – Žák	57
Konstruktory v rámci dědičnosti	58
Modifikátor přístupu protected	60
14. Třída Pedagog a využití výčetového typu	62

Výčtový typ Enum.....	63
Kód enum	64
Použití enum	64
Další metody třídy Pedagog.....	65
Metody: getTitul a setTitul	65
Metoda: Přidej aprobaci.....	66
Metoda: Vrať aprobace.....	66
Metoda: Vrať jméno	66
ETitul – optimalizace výčtového typu	67
Metoda: Vrať info o pedagogovi	69
Metoda: toString třídy Pedagog.....	71
Implementace pedagoga jako třídního.....	71
Ověření funkčnosti.....	72
Zadání	73
15. Ukládání dat do csv souboru, výjimky	74
Class Data	74
AppData, vytvoření vlastní složky programu	75
Tvorba csv souboru.....	77
Csv soubor - formátování	78
Zápis dat do csv souboru	78
Načtení dat z csv souboru	80
Otestování.....	83
16. Uživatelské rozhraní programu.....	86
Definování klíčových příkazů.....	86
Práce s uživatelskými daty.....	87
Vykonávání uživatelských příkazů.....	88
Metoda: Vykonej příkaz	90
Příkaz - Návod.....	91
Příkaz - Nový žák	91
Příkaz - Smaž žáka.....	92
Příkaz – Ukaž třídu	93
Příkaz – Ulož žáky.....	93
Příkaz – Ukonči program.....	93
Závěr.....	95

1. Úvod do objektově orientovaného programování (teorie)

Dostala se Vám do ruky publikace pojednávající o objektově orientovaném programování v jazyku Java. Předpokladem pro čtení je základní znalost syntaxe jazyka a zároveň dovednost práce se základními řídicími strukturami programování – větvením a cykly.

Oproti klasickému strukturovanému programování známému např. z programovacích jazyků C či Pascal se v rámci OOP snažíme o úplně jiný náhled na program jako takový. Budování programu se mnohem více podobá skutečné realitě.

Vysvětleme to na příkladu: Vlastníme automobil. Určitě je skvělé, že si nemusíme náhradní díly vyrábět sami. Že lze zajet do pneuservisu a pořídit si hotovou a otestovanou pneumatiku. Že lze koupit správné brzdové destičky, či nalézt a koupit správně padnoucí světlomet. Můžeme říci, že tyto „objekty“ jsou již někde hotovy a my je nemusíme vyrábět sami. Jen si je pořídíme a můžeme je využívat bez obav, že by bylo něco v nepořádku.

Stejně tak programátoři na celém světě programovali a programují různorodé komponenty, které pak ostatní vývojáři mohou využívat. Zdarma či za úplaty. Dokonce se tvrdí, že téměř vše důležité již bylo někdy v minulosti naprogramováno, takže hlavní úkol tkví v tom klíčové entity vyhledat, naučit se je používat a poskládat je do požadovaného funkčního celku.

To by ale nebyla taková zábava, a proto v této učebnici budeme především programovat od základů vlastní objekty. A to tak, abychom pochopili klíčové vlastnosti a zákonitosti OOP. Tyto pak budeme moci aplikovat i v ostatních objektově orientovaných jazycích, nejen striktně v Javě.

OOP přístup nám zjednodušuje pohled na složitější a delší programy. V případě strukturovaného programování může mít program tisíce řádků v jediném souboru. Stává se tak nepřehledným, a tedy i těžko editovatelným. Oproti tomu objektový přístup pracuje s jednotlivými funkčními celky. Program je pak mnohem přehlednější. Lze se v něm jednoduše vyznat i v případě tisíců řádků kódu.

Důvody použití OOP

Pojďme se opět podívat na příklad. Máme za úkol do programu implementovat informace o různých zaměstnancích. Zřejmě bychom vytvořili několik polí. První pole by sloužilo k ukládání jmen, další pole k ukládání opracovaných let, další pak například k uložení hrubé hodinové mzdy zaměstnance. Informace o jednom daném zaměstnanci by tedy neměly v paměti žádnou souvislost. Spojoval by je pouze index – číslo, pod nímž by byly související informace uloženy v jednotlivých polích. Např. pod indexem 3 by byl uložen Petr Novák, který by měl v dalším poli na indexu 3 uloženo, že pracuje ve firmě již 12 let. V dalším poli týkajícího se mezd by bylo na indexu 3 uvedeno, že zaměstnanec pobírá mzdu 200 Kč hrubého za hodinu.

Uložení složitější entity za použití více polí

String[] jmena	double[] hrubeMzdy	int[] odpracLet
• 1 - Pavel Nesvatba • 2 - Lenka Zorná • 3 - Petr Novák • 4 - Iva Přenosilová	• 1 - 250 • 2 - 358 • 3 - 200 • 4 - 411	• 1 - 7 • 2 - 1 • 3 - 12 • 4 - 5

Z řešení plyne jedna zásadní nevýhoda – pokud by teoreticky v jednom jediném poli došlo k posunu hodnot a v ostatních nikoli, veškeré informace o zaměstnancích by byly nesprávné – dočista promíchané, a tedy nepoužitelné.

Zapouzdření

Nyní řešme výše uvedený příklad pomocí OOP. Nejprve vytvoříme tzv. **třídu** – strukturovaný celek pro uložení zaměstnance jako takového. Obsahuje proměnné pro uložení jména, odpracovaných let i mzdy. Informace jsou tedy uloženy pospolu pod jediným objektem – objektem třídy Zamestanec.java. V tomto případě nám tedy stačí pro potřeby ukládání vytvořit pouze jedině pole – pole pro ukládání objektů typu Zamestanec. Správné informace se budou držet pospolu v rámci jediného objektu a nemůže tedy dojít k promíchání, jako tomu bylo v minulém řešení.

Přidejte si k tomu navíc výhodu plné kontroly nad tím, které vlastnosti zaměstnance budou viditelné a které editovatelné. Pokud existuje mezi položkami v rámci jednoho objektu souvislost, můžete ulehčit uživateli práci a některé funkce zautomatizovat. Kupříkladu hrubá mzda závisí na počtu odpracovaných let. Pokud programovanou třídu upravíte dle tohoto požadavku, hrubá mzda se bude měnit automaticky a uživatel tedy nemůže svou nepozorností způsobit chybu.

Výše popisovaný způsob práce s proměnnými nazýváme **zapouzdřením objektu**.

Zapouzdření objektu

Zamestanec		
String jmeno	double hrubaMzda	int odpracLet

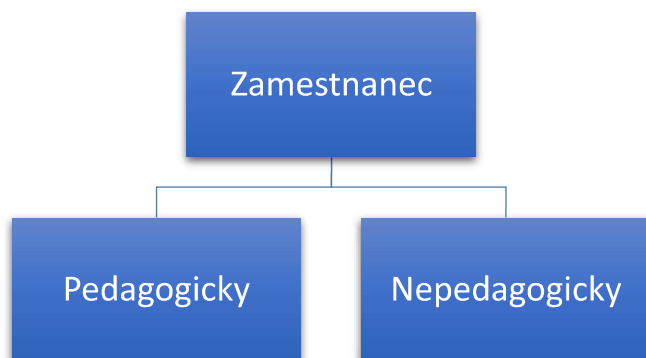
Dědičnost

Nyní si představte, že Váš klient přišel s novým požadavkem. Zaměstnanců je více typů. Např. ve školství zaměstnanci pedagogičtí a nepedagogičtí. Tyto dvě entity se liší minimálně, ale rozdíly zde jsou. Abyste nemuseli objekty znovu programovat či kopírovat části kódu, nabízí OOP další výhodu – třídy lze tvořit na základě jiných, již hotových tříd. V našem případě nejprve definujeme zaměstnance s atributy, které jsou pro všechny typy zaměstnanců stejné. Např. jméno, rodné číslo či hrubá mzda. Následně na základě této třídy definujete zaměstnance pedagogického. Ten zdědí veškeré atributy, ale bude zde přidán atribut aprobace. Nepedagogický zaměstnanec bude zase mít navíc položku pracovní zařazení. Vše ostatní rovněž použije z původní třídy zaměstnanec.

Pojďme poukázat na jednu ne zcela zřetelnou výhodu. Předpokládejme, že bychom chtěli změnit strategii přidělování peněz dle odpracovaných let. Stačilo by tuto funkcionalitu upravit ve třídě zaměstnanec. Třídy pedagogických a nepedagogických zaměstnanců by pak toto chování automaticky převzaly, zdědily by jej od předka. Pro programátora to znamená méně úprav, méně možných chyb a přehlédnutí, méně duplicitního kódu.

Došli jste tedy k dalšímu důležitému pojmu OOP – **dědičnost**.

Dědičnost



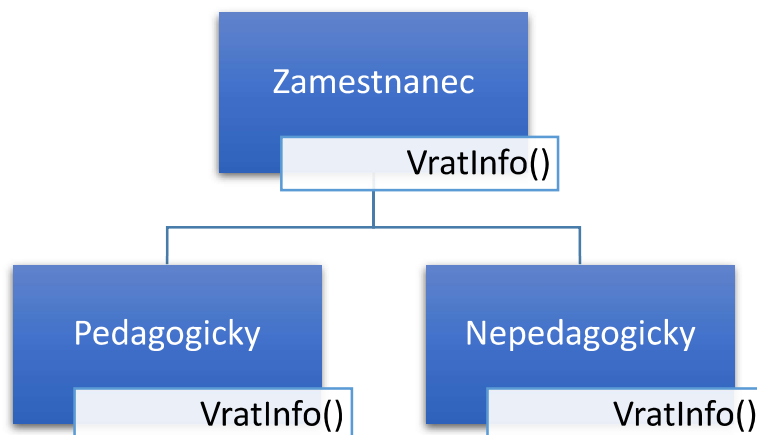
Polymorfismus

Předpokládejme nyní, že dědičnost v našem programu funguje. Jako programátoři chceme, aby všechny třídy v této dědičné struktuře vykonávaly nějaké funkce (lépe řečeno **metody**). Objekty, se kterými se nedá nic dělat, nám velkou službu neudělají. Metody pro výpis by se například mohly nazývat: VratInfoZamestnanec, VratInfoPedagogicky, VratInfoNepedagogicky.

Lépe by však vypadalo, kdyby se metody jmenovaly stejně, např. VratInfo. Představme si tedy 3 metody, které se shodně jmenují, ale každá je v jiném objektu dědičné struktury. V těle každé z těchto 3 metod můžeme naprogramovat libovolný kód ušitý na míru danému objektu.

Výše uvedenou strategii práce s metodami v rámci dědičnosti nazýváme **polymorfismem**. Slovo zní složitě, ale sami během čtení zjistíte, že je to vlastně relativně jednoduchá a hlavně logická vlastnost OOP.

Polymorfismus



Seznámili jsme se tedy se 3 základními vlastnostmi programování objektově orientovaných programů:

- Zapouzdření
- Dědičnost
- Polymorfismus

Nyní můžeme směle pokračovat k další kapitole, kde už budeme i programovat. Všechny vlastnosti uvedené v této kapitole budeme během programování připomínat a brát na zřetel.

Zadání

Pokuste se vlastními slovy vysvětlit pojmy: objektově orientované programování:

- zapouzdření
- dědičnost
- polymorfismus

2. Třída Žák

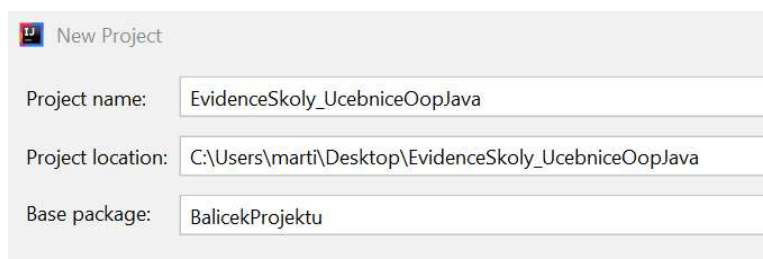
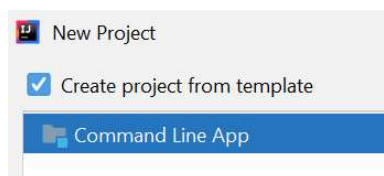
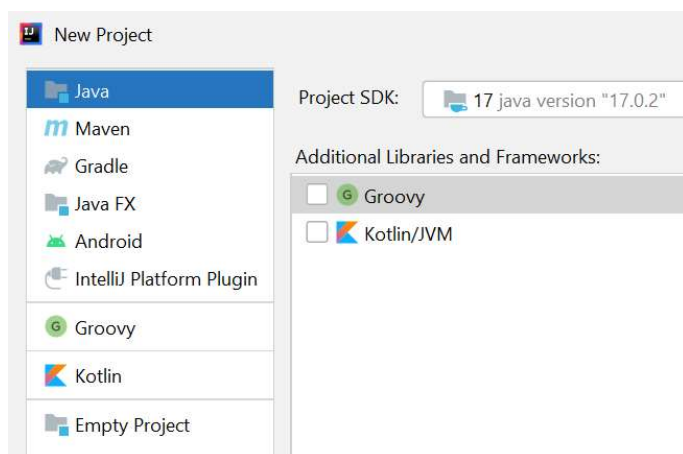
Nyní se již pustíme do samotného programování. Veškeré aspekty OOP v Javě si osvojíme na jediném příkladu, který budeme postupně programovat a upravovat. Vytvoříme evidenci školy.

Program budeme tvořit takovým způsobem, aby co nejlépe vyhovoval nasazení ve školním prostředí a zároveň abychom se z něj co nejvíce naučili. Nebudeme tedy programovat komplexní školní systém vhodný pro plošné nasazení v základním a středním školství, nicméně, jak uvidíte v závěru publikace, program bude funkční i použitelný.

Veškeré screeny pocházejí z programu **IntelliJ IDEA Community edition**. Pokud chcete, můžete k programování použít právě toto prostředí. Nezapomeňte na instalaci **JDK** (Java Development Kit). V tomto případě byl použit balíček OpenJDK 17.0.2.

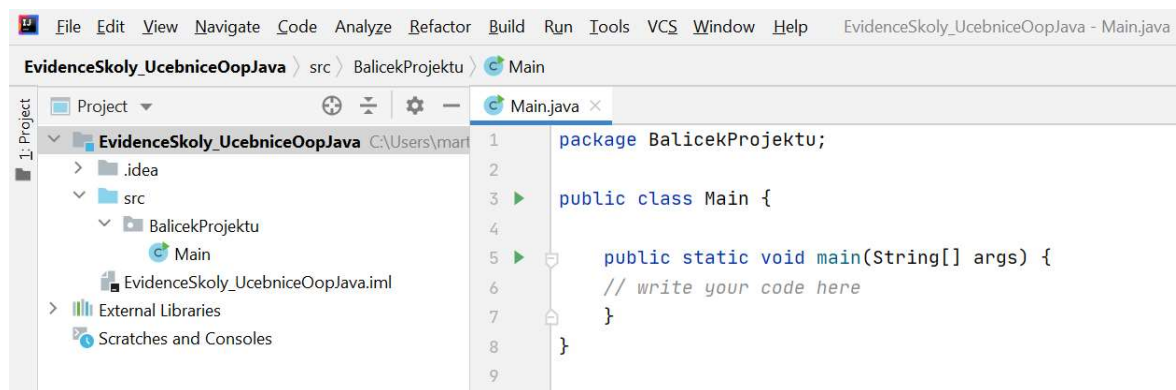
Během zakládání projektu zatrhneme možnost automatické tvorby hlavní třídy programu **Main.java**. Nebudeme ji tak muset tvořit ručně. Pokud chcete, můžete si dle libosti přejmenovat základní balíček, do kterého se budou ukládat všechna Vaše řešení.

Založení projektu



Vytvořili jsme projekt **EvidenceSkoly_UcebniceOopJava**, kde zatím jedinou třídou je hlavní třída **Main.java**. Třída pak obsahuje jedinou metodu – **main**. Kód uvnitř této metody se vykoná právě při spuštění programu. Pokud se vyznáte v tvorbě internetových stránek, je to něco jako soubor **index.html**.

Prostředí IntelliJ a generovaný kód



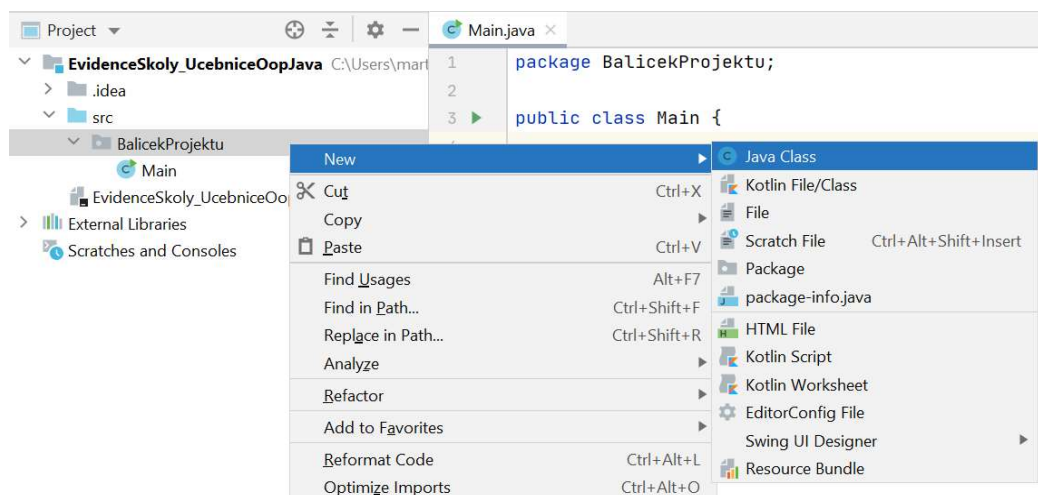
Poznámka 1: Zde je patrný důležitý rozdíl. V Javě názvy tříd vždy začínají velkým písmenem, zatímco názvy metod vždy malým písmenem. Proto jsme schopni v programu rozeznat třídu Main od metody main. V názvech a vůbec v celém programovaném kódu se snažme vyhnout diakritickým znaménkům.

Poznámka 2: Není automaticky dané, že hlavní třída či metoda se automaticky vygeneruje. Každé IDE (vývojové prostředí) se chová jinak. Pokud tomu tak není, tvoříme kód ručně.

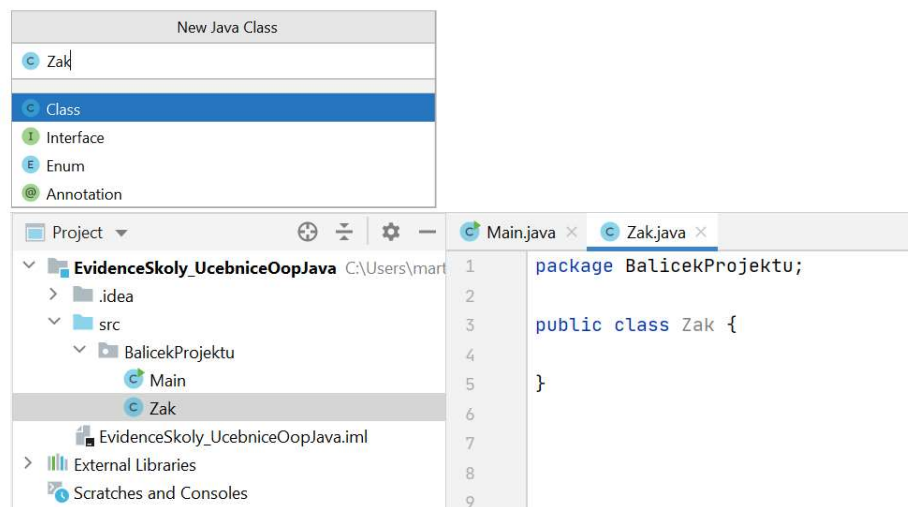
Naše první třída (class)

Pokračujeme a tvoříme naši první vlastní entitu – třídu pro ukládání žáků s názvem **Zak.java**.

Tvorba nové třídy



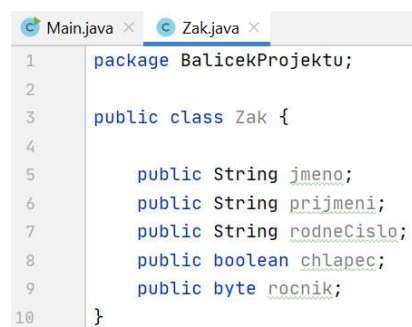
Tvorba nové třídy



Rozmyslíme si atributy, které by měl náš žák mít uloženy. Tvorbu této class vždy přirovnávám k tvoření vstupního papírového formuláře pro nového žáka přijímaného do školského zařízení. Které informace od něj požadujeme, abychom jej mohli plně začlenit do systému školy?

Zdůrazňuji, že zde tvoříme **třídu – předpis**. Teprve až žák náš pomyslný vstupní formulář vyplní a odevzdá, získáváme konkrétní **objekt – instanci dané třídy**.

Atributy třídy Zak (Zak.java)



Komentáře k zápisu atributů

- **String** není jednoduchý datový typ, nýbrž třída (řetězec znaků). Proto je oproti primitivním datovým typům název psán velkým písmenem. S takovými třídami se sem tam během studia potkáme.
- Rodné číslo nemá formát čísla a ani není použito k výpočtům, proto pro jeho ukládání nevolíme int.
- Co se týká pohlaví, náš školský systém stále používá dělení na chlapce a dívky, tedy na 2 pohlaví. Pro jednoduchost jsem tedy zvolil logický datový typ **boolean**, kdy false = dívka a true = chlapec. Šel by samozřejmě použít i jiný způsob, např. výčtový typ.

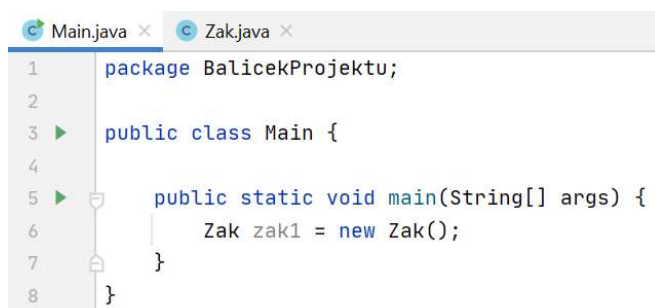
- Pro uložení ročníku jsem se z důvodu úspornosti rozhodl použít datový typ **byte**. Neukládáme vysoká čísla, tedy jeho použití nic nebrání.
- Před každým atributem je psáno klíčové slovo **public**. Jedná se o tzv. **modifikátor přístupu**. Public nám v podstatě říká, že k danému atributu máme odkudkoli z programu přístup, a to jak pro čtení, tak pro zápis. Z jakékoli třídy. Programování nám to značně zjednodušuje, nicméně použití není zcela vhodné. Riskujeme nechtěný či záměrný přepis atributu v paměti. Program pak může padat, vykazovat chyby či nesoulad v uložených informacích.

Pro jednoduchost necháme aktuálně atributy jako public, korekcí se budeme zabývat posléze.

Náš první objekt

Přišel čas zkonstruovat našeho prvního žáka. **Vytvoříme instanci třídy – objekt**. Jak? Přesuneme se do metody **main** třídy Program.java a zavoláme tzv. **konstruktor**. Jedná se o metodu, která právě **z třídy vytváří objekty**. Kde jsme konstruktor vzali? Vznikl automaticky s naší novou třídou Zak, aniž bychom museli cokoli programovat. Přesvědčte se, program nenapíše chybu..

Volání bezparametrického konstrukturu (Main.java)



```
1 package BalicekProjektu;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         Zak zak1 = new Zak();
7     }
8 }
```

Co je ale v objektu zak1 aktuálně uloženo? Co je obsahem jeho atributů? Jsou to prostě defaultní hodnoty jednotlivých datových typů. Nic, co by bylo použitelné v praxi. Objekt zak1 už ale existuje a jeho atributy měnit lze. Umožňuje nám to klíčové slovo public uvedené u každého z atributů. Nejedná se o nejvhodnější postup tvorby objektů, ale jde to.

Nastavení konkrétních hodnot atributům objektu zak1 (Main.java)

```
1 package BalicekProjektu;
2
3 public class Main {
4
5     public static void main(String[] args) {
6         Zak zak1 = new Zak();
7         zak1.jmeno="Patrik";
8         zak1.prijmeni = "Zahálka";
9         zak1.rodneCislo="991230/6789";
10        zak1.chlapec=true;
11        zak1.rocnik = 1;
12    }
13 }
```

V příští kapitole si ukážeme lepší – systematictější postup, jak zkonstruovat nový objekt z naší třídy.

Zadání

Zkuste promyslet, jaké atributy by bylo logické přiřadit objektům:

- Místnost
- Škola
- Zájezdový pobyt

... a dále pomocí jakých datový typů by bylo vhodné tyto atributy definovat.

3. Konstruktory

V minulé kapitole jsme konečně vytvořili první objekt třídy Zak. Učinili jsme tak pomocí defaultního konstruktoru, který každá třída obsahuje, aniž bychom jej museli definovat. Pro plnění atributu jsme použili tečkovou konvenci, např. `zak1.rocnik = 1`.

V této kapitole se podíváme na vhodnější postup při zakládání objektů. Vytvoříme si vlastní konstruktory.

Rovnou si napíšeme, jak by takový konstruktor ve třídě Zak v našem případě vypadal.

Konstruktor třídy Zak (Zak.java)

```
package BalicekProjektu;

public class Zak {

    public String jmeno;
    public String prijmeni;
    public String rodneCislo;
    public boolean chlapec;
    public byte rocnik;

    public Zak(String jmeno, String prijmeni, String rodneCislo, boolean chlapec, byte rocnik) {
        this.jmeno = jmeno;
        this.prijmeni = prijmeni;
        this.rodneCislo = rodneCislo;
        this.chlapec = chlapec;
        this.rocnik = rocnik;
    }
}
```

Pravidla pro tvorbu konstruktoru:

- Píšeme jej dovnitř dané třídy (jako každou jinou metodu).
- V současné fázi výuky by měl být `public` (veřejný), aby bylo možno konstruktor zavolat a objekt tak zkonstruovat i z jiné třídy, především z třídy `Main.java`.
- Má vždy stejný název, jako samotná třída (včetně malých a velkých písmen), v našem případě `Zak`.
- Do závorky v hlavičce konstruktoru píšeme seznam proměnných, pomocí kterých chceme objekt definovat. Mohou to být všechny atributy (jako v případě našeho příkladu) ale také nemusí. Pro ilustraci – případný atribut `prumerZnamek` dává smysl plnit automaticky dle dostupných hodnocení, nikoli hodnotou natvrdo.
- Ve složených závorkách je zpravidla umístěn kód, kdy hodnoty **formálních proměnných** (z hlavičky konstruktoru) ukládáme do **skutečných proměnných** daného objektu (ty jsou pro odlišení vždy označeny klíčovým slovem **this**).

Poznámka: Pokud bychom se chtěli vyhnout použití klíčového slova `this`, je možné názvy formálních a skutečných proměnných od sebe odlišit jiným způsobem, například pomocí podtržítka.

Alternativní zápis atributů a konstruktoru (Zak.java)

```
package BalicekProjektu;

public class Zak {

    public String _jmeno;
    public String _prijmeni;
    public String _rodneCislo;
    public boolean _chlapec;
    public byte _rocnik;

    public Zak(String jmeno, String prijmeni, String rodneCislo, boolean chlapec, byte rocnik) {
        _jmeno = jmeno;
        _prijmeni = prijmeni;
        _rodneCislo = rodneCislo;
        _chlapec = chlapec;
        _rocnik = rocnik;
    }
}
```

Konstruktor máme hotov a lze jej tedy zavolat z Program.java a vytvořit tak nového žáka.

Volání konstrukturu (Main.java)

```
public static void main(String[] args) {
    Zak zak2 = new Zak( jmeno: "Jana", prijmeni: "Výborná", rodneCislo: "115111/5284", chlapec: false, (byte)2);
}
```

Poznámka: Všimněte si nutnosti přetypovat číslo ročníku na byte.

Přetížený konstruktor

Je pravdou, že ve škole drtivé množství nově přidávaných žáků nastupuje právě do 1. ročníku. Je tedy zbytečné, aby tuto hodnotu uživatel byl nucen na vstupu neustále dokola opisovat.

Existuje zajímavé řešení, kdy vytvoříme ještě jeden náš konstruktor, ale na vstupu vynecháme právě ročník, takže uživatel jej nebude muset zadávat. V těle metody pak automaticky přiřadíme do proměnné hodnotu 1.

Příklad přetíženého konstruktoru (Zak.java)

```
public Zak(String jmeno, String prijmeni, String rodneCislo, boolean chlapec) {  
    this.jmeno = jmeno;  
    this.prijmeni = prijmeni;  
    this.rodneCislo = rodneCislo;  
    this.chlapec = chlapec;  
    this.rocnik = 1;  
}
```

Získali jsme druhý konstruktory který se stejným jménem, ale liší se ve vstupních parametrech. Tuto metodu nazýváme **přetíženým konstruktorem**.

Odstranění duplicitního kódu v rámci přetíženého konstruktoru

V programování, jak známo, příliš nefandíme duplicitnímu kódu. Při ladění programu přináší jen komplikace. V našem případě stejné bloky konstruktů tedy nebudeme kopírovat. Budeme se na ně pouze odkazovat. Konstruktory upravíme následujícím způsobem..

Odstranění duplicity dat v konstruktorech (Zak.java)

```
//Konstruktor  
public Zak(String jmeno, String prijmeni, String rodneCislo, boolean chlapec, byte rocnik) {  
    this.jmeno = jmeno;  
    this.prijmeni = prijmeni;  
    this.rodneCislo = rodneCislo;  
    this.chlapec = chlapec;  
    this.rocnik = rocnik;  
}  
  
//Přetížený konstruktor  
public Zak(String jmeno, String prijmeni, String rodneCislo, boolean chlapec) {  
    this(jmeno, prijmeni, rodneCislo, chlapec, (byte)1);  
}
```

Poznámka: Slovo „this“ nahrazuje název třídy, ve které se aktuálně nacházíme – Zak.

Nyní můžeme konstruktory jednoduše zavolat z třídy Program.java. Vytvoříme si jednoho prvňáka.

Volání přetíženého konstruktoru (Main.java)

```
Zak zak3 = new Zak( jmeno: "Jindřich", prijmeni: "Ospalý", rodneCislo: "981129/1235", chlapec: true);
```

Přetížených konstruktorů (a tedy možností, jak zkonstruovat objekt) může být velké množství. Představte si situaci, kdy například chcete žáka vytvořit, ale nemáte k němu veškeré informace. Například neznáme mailovou adresu či telefonní kontakt na žáka. To sice není náš případ, nicméně často po nás může klient požadovat, aby bylo možné objekt vytvořit jen na základě omezené množiny údajů.

Zrušení implicitního konstruktoru

Jistě Vám neunikla nepříjemná skutečnost, že zatímco objekty `zak2` a `zak3` vytvořené pomocí nových konstruktorů jsme bez problémů vytvořili, `zak1` nyní nově vykazuje chybu. Je to dáno faktem, že v okamžiku vytvoření byť jen jediného vlastního konstruktoru ten implicitní přestane fungovat. Kompilátor ho již nevytvoří automaticky, jako tomu bylo na začátku.

Pokud bychom chtěli z jakéhokoli důvodu tento „prázdný“ konstruktor zachovat, není problém jej do třídy `Zak` doprogramovat. Vytvoříme tím pouze další z přetížených konstruktorů. Konstrukce je jednoduchá.

Bezparametrický konstruktor (Zak.java)

```
public Zak() {  
}
```

Nyní známe pojem konstruktor, víme, jak ho naprogramovat a jak jej zavolat k vytvoření nového objektu. Známe pojmy implicitní a přetížený konstruktor.

V další kapitole se pokusíme přidat do naší nové třídy `Zak` další, již existující třídu.

4. Datum a čas

V této kapitole se pokusíme do naší třídy `Zak` přidat informaci o jeho datu narození. Situaci nebudeme řešit třemi celočíselnými proměnnými pro uložení dne, měsíce a roku. Budeme chytřejší -využijeme k tomu již vytvořenou třídu. Ta nám zároveň otevře další možnosti, jak s datem pracovat.

Předdefinovaných tříd využitelných pro ukládání data a času je v Javě vytvořeno mnoho a rovněž se jich mnoho používá. My budeme pracovat s balíčkem **LocalDate**. Je programátory prověřený a zároveň pro nás dobře použitelný.

Postup implementace `LocalDate` do našeho programu

1. Ve třídách `Zak.java` a `Main.java` importujeme balíček **java.time.LocalDate**, a to pomocí klíčového slova **import**.
2. V `Main.java` importujeme knihovnu **java.time.Month**, která nám umožní pracovat s výčtem měsíců v roce.
3. Do naší třídy `Zak` přidáme položku `datumNarozeni` typu **LocalDate**.
4. Položku přidáme i do všech přetížených konstruktorů kromě konstruktoru bezparametrického.

LocalDate (Zak.java)

```
package BalicekProjektu;
import java.time.LocalDate;

public class Zak {

    public String jmeno;
    public String prijmeni;
    public String rodneCislo;
    public LocalDate datumNarozeni;
    public boolean chlapec;
    public byte rocnik;

    //Konstruktor
    public Zak(String jmeno, String prijmeni, String rodneCislo, LocalDate datumNarozeni, boolean chlapec, byte rocnik) {
        this.jmeno = jmeno;
        this.prijmeni = prijmeni;
        this.rodneCislo = rodneCislo;
        this.datumNarozeni=datumNarozeni;
        this.chlapec = chlapec;
        this.rocnik = rocnik;
    }

    //Přetížený konstruktor
    public Zak(String jmeno, String prijmeni, String rodneCislo, LocalDate datumNarozeni, boolean chlapec) {
        this(jmeno, prijmeni, rodneCislo, datumNarozeni, chlapec, (byte)1);
    }

    //Bezparametrický konstruktor
    public Zak() {
    }
}
```

Jistě jste si všimli vzniklých chyb. Stačí však ve třídě Main.java upravit volání konstruktorů vložením informace o datu narození. Nemůžeme přece náš nově změněný 6ti parametrický konstruktor volat pouze pomocí 5ti parametrů. Možností jak to udělat je více, níže uvádím pár z nich.

Volání upravených konstruktorů Zak (Main.java)

```
package BalicekProjektu;
import java.time.LocalDate;
import java.time.Month;

public class Main {

    public static void main(String[] args) {

        //zak1
        Zak zak1 = new Zak();
        zak1.jmeno="Patrik";
        zak1.prijmeni = "Zahálka";
        zak1.rodneCislo="991230/6789";
        LocalDate ld1 = LocalDate.of( year: 2000, month: 4, dayOfMonth: 28);
        zak1.datumNarozeni=ld1;
        zak1.chlapec=true;
        zak1.rocnik = 1;

        //zak2
        LocalDate ld2 = LocalDate.of( year: 2001, Month.MARCH, dayOfMonth: 20);
        Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);

        Zak zak3 = new Zak("Jindřich", "Ospalý", "981129/1235", LocalDate.of( year: 2002, month: 12, dayOfMonth: 14), true);
    }
}
```

Z kódu je patrné, že nová instance třídy LocalDate se netvoří pomocí klasického volání konstruktoru klíčovým slovem **new**, nýbrž voláním tovární metody **of()**.

*Poznámka: Pokud by Vaším zájmem bylo získat datum aktuální, potom stačí napsat jednoduchý příkaz **LocalDate datumDnesni = LocalDate.now();***

Výpis data

K formátování výstupu data budeme používat **statickou** třídu **DateTimeFormatter**, z ní pak použijeme **statickou** metodu **ofLocalizedDate** pro převod data do lokální podoby.

Poznámka: Vysvětlováním statiky se budeme zabírat v některé z dalších kapitol. Prozatím nám postačí vědět, že statické třídy existují v programu pouze jednou, netvoří se z nich tedy instance pomocí konstruktoru.

Pomocí **výčtového typu** `FormatStyle` můžeme zvolit vzhled výpisu data, jaký nám vyhovuje nejvíce. Příkládám příklad kódu včetně samotného výpisu na consoli.

Možnosti formátování pro výpis data (Main.java)

```
package BalicekProjektu;
import java.time.LocalDate;
import java.time.Month;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;

public class Main {

    public static void main(String[] args) {

        //zak2
        LocalDate ld2 = LocalDate.of( year: 2001, Month.MARCH, dayOfMonth: 20);
        Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);

        System.out.println(ld2.format(DateTimeFormatter.ISO_DATE));
        System.out.println(ld2.format(DateTimeFormatter.ofLocalizedDate((FormatStyle.FULL))));
        System.out.println(ld2.format(DateTimeFormatter.ofLocalizedDate((FormatStyle.LONG))));
        System.out.println(ld2.format(DateTimeFormatter.ofLocalizedDate((FormatStyle.MEDIUM))));
        System.out.println(ld2.format(DateTimeFormatter.ofLocalizedDate((FormatStyle.SHORT))));
```



```
Run: Main x
2001-03-20
úterý 20. března 2001
20. března 2001
20. 3. 2001
20.03.01
```

Poznámka: Na vysvětlenou - výčtový typ je programátorem vytvořený datový typ, který může nabývat pouze omezeného množství hodnot. Např. výčtový typ den by mohl nabývat hodnot: po, ut, st, ct, pa, so, ne. Tomuto tématu se budeme podrobněji věnovat v kapitole číslo 14.

Některým z Vás možná vrtá hlavou, že v našem příkladu nemusí korespondovat datum narození a pohlaví s rodným číslem (neboť logická vazba existuje). Vězte, že k tomuto ověřování se dostaneme. Ale hezky popořadě. Nejprve si musíme něco říci o tvorbě vlastních metod. Až budeme schopni tvořit vlastní metody, vytvoříme i metody pro validaci rodného čísla.

5. Metody (teorie)

Pokud by naše třídy obsahovaly pouze proměnné, fungovaly by maximálně jako jakýsi aparát pro ukládání hodnot. My ale potřebujeme, aby náš program ožil. Uměl s proměnnými pracovat. Měnil jejich hodnotu, vypisoval, ověřoval jejich správnost, počítal s nimi, ovlivňoval běh programu a mnoho dalšího. Správné konstruování metod je pro běh programu nezbytné. V podstatě jimi říkáme, které operace s objektem (třídou) či proměnnými lze dělat a které naopak nikoli. Časté jsou i metody vykonávající určité funkce bez navázání na stav vnitřních proměnných.

Každá metoda obsahuje 2 důležité části – hlavičku a tělo:

- Hlavička obsahuje název metody a definuje, s jakými typy hodnot bude metoda pracovat
- Tělo obsahuje příkazy, které se mají při zavolání metody provést, případně určuje, která hodnota z metody bude vystupovat jako výsledek.

Na začátku programování je nasnadě si zapřemýšlet, jaké přesně operace od naší metody očekáváme.

Konstrukce hlavičky

- Vymyslíme název, který metodu co možná nejlépe vystihuje. První písmeno názvu metody se v Javě uvádí **malým písmenem**.
- **Modifikátorem přístupu public/private** určíme, zda bude možno volat metodu i z jiné třídy, než např. ze třídy Main.java.
- Pokusíme se přesně vystihnout, které hodnoty metoda potřebuje pro svou činnost (**určíme vstupy**). **Pozor! Za vstupní nepovažujeme ty hodnoty, které jsou již v objektu definovány – hodnoty, které má metoda již k dispozici!**
- Uvážíme, zda má metoda vracet nějakou hodnotu či nikoli (**výstup**). Výstupem může být buď datový typ, objekt nebo nic (**void**).

Poznámka: Výjimku tvoří konstruktory. U nich výstup nedefinujeme vůbec, respektive výstupem je objekt samotný.

Obecný tvar zápisu hlavičky

Private/public	výstupní datový typ	Název metody	(vstupní parametry s datovými typy)
----------------	---------------------	--------------	-------------------------------------

*Poznámka: Pokud v rámci dědičnosti naše metoda existuje již v předkovi, pro její přepis doplníme klíčové slovo **@Override**. Takto učiníme již před samotnou hlavičkou metody. Informace se týká **polymorfismu**, v praxi si jej ukážeme později.*

Tělo metody

- Píšeme do **složených závorek**.
- V případě požadované návratové hodnoty tuto hodnotu označíme v kódu klíčovým slovem **return**.
Pozor - po přečtení příkazu return se vyskočí z metody pryč a další kód metody se již nevykoná. Proto „return“ řádek bývá zpravidla posledním řádkem metody.

Dělení metod dle vstupů a výstupů

- **Má vstupní parametry i výstup**
Hodí se např. na matematické operace.
- **Má pouze vstupní parametry**
Nejčastěji pro zápis (do proměnné objektu, do seznamu, do souboru, databáze...)
- **Má pouze výstup**
Nejčastěji pro čtení (z proměnné, seznamu, souboru, ...)
- **Nemá vstupní parametry ani výstup**

Příklady použití metod

Představme si třídu Zprava, jejímž jediným atributem je textová proměnná `uvitaciZprava`. Proměnná je modifikátorem přístupu označena jako **private**. Není tedy možné z ní číst nebo do ní zapisovat skrze `Main.java`. S proměnnou lze manipulovat pouze pomocí veřejných metod.

Metody v testovací třídě Zprava.java

```
public class Zprava {  
    private String uvitaciZprava;  
  
    //Příklad metody, která má vstupní parametry i výstup  
    public String vratUvitaciZpravuS0slovenim(String jmeno) {  
        return String.format("{0}, {1}", jmeno, uvitaciZprava);  
    }  
  
    //Příklad metody, která má pouze vstupní parametry  
    public void nastavUvitaciZpravu(String text) {  
        uvitaciZprava = text;  
    }  
  
    //Příklad metody, která má pouze výstup:  
    public String vratUvitaciZpravu() {  
        return uvitaciZprava;  
    }  
  
    //Příklad metody, která nemá vstupní parametry ani výstup:  
    public void smazUvitaciZpravu() {  
        uvitaciZprava = "";  
    }  
}
```

Pro pořádek uvádím i kód volání testovacích metod z Main.java.

Volání testovacích metod (Main.java)

```
public static void main(String[] args) {  
  
    Zprava zprava = new Zprava();  
    zprava.nastavUvitaciZpravu( text: "Hi");  
    System.out.println(zprava.vratUvitaciZpravu());  
    System.out.println(zprava.vratUvitaciZpravuS0slovenim( jmeno: "Martin"));  
    zprava.smazUvitaciZpravu();  
}
```

Snad jste si během čtení kapitoly udělali jasný obrázek o tvorbě metod, o různých typech metod a o jejich syntaktickém zápisu. Správný myšlenkový proces tvorby metod si pečlivě zafixujte. Je klíčový pro kvalitní, správnou a logickou tvorbu našich OOP programů.

Zadání

VeźmĚte si papír a udĚlejte rozvahu nad tĚm, kterĚ metody pro prĚci s objektem typu Zak by bylo vhodnĚ vytvořit pro potřeby našeho projektu Evidence školy. Zkuste takových funkcionalit najít co nejvíce.

NalezenĚ metody vhodnĚ pojmenujte a promyslete, jakĚ vstupy a vĚstupy by mĚly obsahovat. UvidĚme, v kolika z nich se budeme v prĚjĚších kapitolĚch shodovat.

6. Metody pro výpis informací

V této kapitole si naprogramujeme metody vypisující či vracející informace o objektech. Zatím dokážeme objekt vytvořit (pomocí konstruktoru), ale nemáme důkaz, že se tak opravdu děje. Z paměti jsme se informace o objektu ještě nepokoušeli přechít.

Metoda vypisInfo

Metoda bude vypisovat přímo na konzoli informace o žákovi, které nám přijdou důležité. Není nezbytné vypisovat veškeré informace z objektu. Přizpůsobujeme se zadání či přání klienta.

Řekněme si, že je pro nás zajímavý výpis ve formátu: „**Jana Výborná (28. 4. 2002), 3. ročník**“:

- Metoda nebude mít žádný vstup, neboť objekt má všechna data potřebná pro výpis uložena v sobě (má je k dispozici).
- Metoda nebude mít žádný výstup, neboť pouze vykonává danou činnost – posílá informace k výpisu rovnou na konzoli.

Metoda vypisInfo (Zak.java)

```
public void vypisInfo()
{
    String text="";
    text+=String.format("%s %s ",jmeno, prijmeni);
    text+="("+datumNarozeni.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM))+"), ";
    text+=rocnik+". ročník";
    System.out.println(text);
}
```

*Poznámka: Pro potřeby výpisu data nezapomeneme nahoru do Zak.java importovat balíček pro práci s formátem data, tedy **import java.time.format.***;*

Volání metody vypisInfo (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001,Month.MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284",ld2, false,(byte)2);
zak2.vypisInfo();
```



Metoda tedy funguje dle našich požadavků, má ale jeden zásadní nedostatek. Kdykoli bychom informace o objektech typu `Zak` chtěli vypisovat do jiného prostředí než konzolového, narazili bychom na limit námi konstruované třídy, potažmo metody.

Lepší variantou je zařídit, aby metoda pouze vrátila požadovaný text. Až posléze ve třídě `Main.java` by se tak rozhodlo, jak bude s výpisem naloženo, kde přesně bude text poslán či zobrazen.

Metoda sama o sobě nic vypisovat nebude, bude pouze vracet text, proto změníme název. Místo `vypisInfo` ji pojmenujeme `vratInfo`. Do metody opět nemusí nic vstupovat, výstup zde ale bude, a to ve formě textu. Připomínám, že v těle metody je návratová hodnota určena klíčovým slovem **return**.

Metoda `vratInfo` (`Zak.java`)

```
public String vratInfo()
{
    String text="";
    text+=String.format("%s %s ",jmeno, prijmeni);
    text+=("+"+datumNarozeni.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM))+"), ";
    text+=rocnik+" . ročník";
    return text;
}
```

Až při volání metody v `Main.java` určíme, že vrácený text bude zobrazen v konzoli.

Volání metody `vratInfo` (`Main.java`)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month.MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
System.out.println(zak2.vratInfo());
```



Je zřejmé, že metody `vypisInfo` i `vratInfo` dělají totéž. Pro širší použitelnost třídy však striktně doporučuji vyhnout se konstruování metod typu `vypisInfo`. Proto tuto metodu v kódu zakomentujte, nebo ji úplně smažte. Je výhodnější nechat výpis textu na starosti hlavní třídy `Main.java`.

Metoda `toString`

Metoda **`toString`** je v OOP hojně používaná. Umožňuje nám vracet text stejně jako výše uvedená metoda `vratInfo()`. Je zde však jeden zásadní rozdíl. Metoda `vratInfo` je naší původní metodou. Oproti tomu

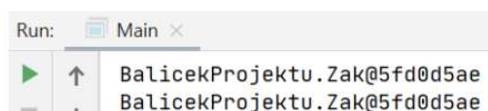
metodu `toString` obsahuje již od počátku každý objekt. Ano, i náš objekt typu `Zak` ji obsahuje, aniž bychom ji ve třídě sami definovali. Jak je to možné? Díky dědičnosti.

V Javě existuje jediná hlavní třída s názvem **Object.java**. Její součástí je, jak asi správně tušíte, i zmiňovaná metoda `toString`. Pokud u Vaší třídy neoznačíte jiný dědičný vztah, je dáno, že jejím přímým rodičem se stává právě třída `Object`. A i kdybyste dědili z jiné třídy, tak ta má jako předka rovněž `Object.java`. Tedy metodě `toString` se v podstatě nedá vyhnout. Je nedílnou součástí každé třídy, ať chceme či nechceme.

Metoda `toString` má v defaultním nastavení za úkol vracet textovou definici daného objektu – **název třídy** objektu a **hash code** objektu. Na obrázku níže vidíte důkaz – metodu `toString` jsme ve třídě `Zak` nedefinovali, přesto nám ale program při jejím volání nevypisuje chybu, nýbrž textovou definici objektu `zak2`.

Volání metody `toString` (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month: MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
System.out.println(zak2.toString());
System.out.println(zak2);
```



Dále je z obrázku vidět, že zde existuje **implicitní konverze** - pokud budeme chtít vypsat informace o objektu (zde pomocí `System.out.println`), kompilátor Javy metodu `toString` na objektu zavolá. Přesto, že ji v zápisu příkazu nemáte uvedenu. Oba výpisy na konzoli jsou totožné.

Pokud byste chtěli metodu `toString` používat, ale její defaultní výpis je Vám k ničemu, lze chování této metody přepsat. Stačí ji ve vlastní třídě naprogramovat a před definicí použít zápis **@Override**.

Osobně jsem si zvyknul používat metodu `toString` tak, že mi vrací typ objektu a hodnoty všech svých vnitřních proměnných. IntelliJ IDEA má pro tyto účely pomocný nástroj, který se Vám jistě bude velmi zamlouvat. Generuje totiž chování metody `toString` automaticky:

- Umístěte kurzor do třídy `Zak.java`.
- Stiskněte klávesovou zkratku levý `alt`+`insert`.
- Pomocí kurzorových kláves vyberte `toString()`.
- V dalším zobrazeném okně ponechte vybrané všechny atributy v seznamu.
- Dole nechte zatrženou položku `insert @Override`.
- Stiskněte `OK` nebo `Enter` a kochejte se hotovým kódem.

Metoda toString - generovaný kód v prostředí IntelliJ (Zak.java)

```
@Override
public String toString() {
    return "Zak{" +
        "jmeno='" + jmeno + '\'' +
        ", prijmeni='" + prijmeni + '\'' +
        ", rodneCislo='" + rodneCislo + '\'' +
        ", datumNarozeni='" + datumNarozeni +
        ", chlapec=" + chlapec +
        ", rocnik=" + rocnik +
        '}';
}
```

Volání přepsané metody toString (Main.java)

```
//zak2
LocalDate ld2 = LocalDate.of( year: 2001, Month.MARCH, dayOfMonth: 20);
Zak zak2 = new Zak("Jana", "Výborná", "115111/5284", ld2, false, (byte)2);
System.out.println(zak2.toString());
System.out.println(zak2);
```

Run: Main ×

```
 Zak{jmeno='Jana', prijmeni='Výborná', rodneCislo='115111/5284', datumNarozeni=2001-03-20, chlapec=false, rocnik=2}
 Zak{jmeno='Jana', prijmeni='Výborná', rodneCislo='115111/5284', datumNarozeni=2001-03-20, chlapec=false, rocnik=2}
```

V následující kapitole si zkusíme pro potřeby našeho programu vytvořit několik dalších metod.

Zadání

Naprogramujte a zavolejte metodu VratInfo2(), která vrací text ve formátu například:

„Jana Výborná – dívka, nematuritní ročník“

nebo pro jiný objekt

„Pavel Jedničkář – chlapec, maturitní ročník“

Předpokládejme, že maturitním ročníkem máme na mysli právě 4. ročník.